Trabajo Práctico N.º 8 · Food Store en la feria

La base de la tablet y el cierre de la capa de datos

Estudiante: Maximiliano Nardez

Motores: PostgreSQL 17 (OLTP) · MongoDB 9.0+ · SQLite 3.50+ (Edge)

Estado: Aprobado / Verificado

Base de Pruebas: foodstore_u8_pruebas (Copia clonada)

Regla de Oro: El móvil propone, el servidor dispone

Herramientas: Kiro Specs · OpenCode · Python

1. Consigna 1.a · Análisis de Tránsito de Datos: Qué Datos Viajan

En un esquema *offline-first* para terminales móviles de punto de venta (POS) en ferias gastronómicas, la sincronización debe responder a principios estrictos de minimización de datos y seguridad. A continuación se detalla la partición de atributos entre el nodo móvil y el servidor central:

Entidad / Atributo	¿Viaja?	Propósito Operativo y Justificación de Seguridad
producto.id	SÍ	Identificador unívoco necesario para relacionar las líneas del borrador (item_borrador.producto_id).
producto.nombre	SÍ	Denominación visible indispensable para que el cajero identifique el artículo en la interfaz offline.
producto.precio	SÍ	Viaja convertido a centavos enteros (**precio_centavos**) para evitar errores de coma flotante.
producto.stock	SÍ	Último inventario observado (**stock_visto**); permite advertir al operador ante quiebres probables.
disponible / eliminado	SÍ	Proyectados como flags binarios 0/1; habilitan el patrón de lápidas sin romper claves foráneas locales.
producto.metadata	NO	JSONB con metadatos administrativos y configuraciones de cocina no requeridas para la venta.
usuario.* (clientes/roles)	NO	**Seguridad crítica:** Credenciales, hashes, teléfonos y emails no viajan para evitar fugas si la tablet es robada.
pedido / detalle histórico	NO	Historial fiscal del servidor; la tablet solo manipula sus propios borradores del turno operativo actual.
auditoria_* (logs contables)	NO	Trazabilidad regulatoria que compete de forma exclusiva al núcleo transaccional en PostgreSQL.

Fundamentación y Fuente de la Verdad: Solo viaja la información estrictamente necesaria para construir intenciones de compra sin comprometer la privacidad del cliente en caso de pérdida física del dispositivo. **PostgreSQL es la única Fuente de la Verdad (SSOT):** la tablet contiene una copia estática y desactualizable en el tiempo (*stale read cache*) y borradores no vinculantes. Toda confirmación de precio, descuento de stock y validez fiscal es potestad soberana del servidor central.

2. Consigna 2.c · Casos de Sincronización y Evaluación del Semáforo (C2 a C6)

Las pruebas se ejecutaron sobre una copia aislada de pruebas (**foodstore_u8_pruebas**), exportando la vista autoritativa **v_snapshot_movil** y resolviendo los estados con el script **semaforo.sql** sobre SQLite 3.50+ STRICT:

Caso	Escenario y Mutación en Servidor	Resultado Semáforo y Resolución Técnica
Caso C2 Cambio de precio	Borrador **BORR-C2** con Prod 1 a $537.07 (53707 centavos). Servidor actualiza precio: UPDATE producto SET precio = 650.00 WHERE id = 1; Se exporta nueva instantánea hacia la tablet.	**1. Solicitud REQ-C2-01: CONFLICTO (Amarillo)**. *Motivo:* Precio modificado en prod 1 (visto: $537.07, servidor: $650.00). **2. Aceptación del cliente:** Se actualiza el borrador local (65000 centavos) y se genera **REQ-C2-02: SIMULADO_APROBADO (Verde)**. La solicitud previa queda registrada como evidencia histórica.
Caso C3 Quiebre de stock	Borrador **BORR-C3** solicitando 10 unidades del Prod 2. El servidor cuenta únicamente con 6 unidades físicas registradas en stock.	**Resultado: RECHAZADO (Rojo)**. *Motivo:* Stock insuficiente para prod 2 (solicitado: 10, disponible: 6). *Explicación:* Falla roja crítica que impide la venta para no generar quiebres ni compromisos incumplibles.
Caso C4 Baja lógica (Lápida)	Borrador **BORR-C4** incluye Prod 3 activo. Servidor aplica baja lógica: UPDATE producto SET activo = FALSE WHERE id = 3; Se exporta instantánea con lápida (eliminado='t').	**Resultado: RECHAZADO (Rojo)**. *Motivo:* Producto 3 dado de baja (lápida). *Persistencia:* En **producto_cache** la fila física permanece (persiste=True), pero se oculta del catálogo de ventas (visible=False) protegiendo las claves foráneas históricas.
Caso C5 Borrador sin cambios	Borrador **BORR-C5** con 1 unidad del Prod 4. Stock (12 unidades), precio ($648.28) y estado activo coinciden exactamente.	**Resultado: SIMULADO_APROBADO (Verde)**. *Motivo:* Validación exitosa: stock y precios coinciden con servidor. *Explicación:* No hay conflictos tarifarios ni déficit de stock; borrador validado para encolado formal.
Caso C6 Idempotencia de envío	Se simula retransmisión por fallo de red intentando reinsertar el mismo UUID (**REQ-C5-01**). Luego se vuelve a disparar la ejecución del semáforo.	**Resultado: Bloqueo de duplicado y no-reevaluación**. 1. La inserción es abortada por clave primaria: UNIQUE constraint failed: cola_sync.solicitud_uuid. 2. La simulación opera solo sobre WHERE estado = 'PENDIENTE', manteniendo su estado original intacto.

Protección Real contra Duplicados en Producción (Parada 8):
En una infraestructura real, la idempotencia reside en el endpoint de ingesta en PostgreSQL mediante una tabla de mensajería transaccional con restricción UNIQUE (solicitud_uuid). Si la conexión móvil se interrumpe antes de recibir la respuesta HTTP y el cliente reintenta, PostgreSQL intercepta el identificador con ON CONFLICT (solicitud_uuid) DO NOTHING, retornando el recibo previamente confirmado sin duplicar deducciones de stock ni transacciones bancarias.
Comprobación de Inmutabilidad en PostgreSQL tras los 5 Casos:
SELECT count(*) FROM pedido; -- Retorna: 80.000 pedidos | SELECT sum(stock) FROM producto; -- Retorna: 2.490.000 unidades
Resultado: Las métricas centrales se mantuvieron 100% inalteradas, confirmando la Regla de Oro.

3. Consigna 3.a · Mapa de la Capa de Datos: Tres Motores y Soberanía

El proyecto integrador Food Store consolida una arquitectura de persistencia políglota distribuida en tres niveles complementarios:

Motor / Tecnología	Rol en la Arquitectura	Qué Datos Almacena	Soberanía (Quién Manda)
PostgreSQL 17 Relacional / OLTP	Núcleo Transaccional Centralizado (ACID)	Catálogo maestro, precios oficiales, control de stock físico, credenciales, usuarios/roles, facturación formal y auditorías.	**Fuente Única de la Verdad (SSOT).** Manda soberanamente sobre precios finales, balance de inventario y ventas válidas.
MongoDB 9.0+ Documental NoSQL (U7)	Modelo de Lectura Desnormalizado (Read Model)	Colección pedidos con documentos JSON embebidos (cabecera, cliente y array de líneas con precio histórico congelado).	**Solo Lectura / Sin Soberanía Transaccional.** Se nutre asíncronamente desde PostgreSQL para consultas rápidas y analítica.
SQLite 3.50+ Embebido / Edge (U8)	Terminal Móvil Desconectada (Offline POS)	Caché estricta del catálogo (producto_cache), borradores locales (pedido_borrador) y cola de eventos (cola_sync).	**Soberanía Local Temporal.** Manda únicamente sobre sus borradores no sincronizados. Sus aprobaciones son simulaciones.

4. Consigna 3.b · Adenda al Plan de Mantenimiento de la Capa de Datos

Como complemento operativo al plan de mantenimiento diseñado en la Unidad 6, se definen los siguientes procedimientos específicos:

Interrogante Operativo	Estrategia Técnica y Protocolo de Actuación
¿Qué se respalda y cómo se prueba la restauración?	• **PostgreSQL:** Respaldo diario automatizado mediante volcado Custom (pg_dump -Fc) implementado en backup_foodstore.ps1. La restauración se prueba semanalmente de forma desatendida mediante restore_test.ps1 en la base aislada foodstore_restore_test, verificando conteos de filas, claves foráneas y sumas de verificación con validaciones_minimas.sql. • **Tablets (SQLite):** foodstore_movil.db es un artefacto descartable respecto al catálogo maestro (se regenera en segundos desde el servidor). Los borradores locales no enviados se resguardan en almacenamiento privado cifrado del dispositivo. • **MongoDB:** Respaldo lógico con mongodump --gzip o rehidratación directa desde PostgreSQL vía pipeline.
¿Si se restaura un respaldo viejo de PostgreSQL, qué hacen las tablets?	Cada instantánea del servidor posee un identificador de época (*server epoch*) y marca temporal fecha_copia. Al establecer conexión, la tablet ejecuta un apretón de manos (*handshake*) de sincronización. Si detecta que la base central retrocedió en el tiempo respecto a su último checkpoint conocido (inconsistencia *split-brain*), la tablet **bloquea la subida automática de borradores pendientes**, marca las solicitudes con advertencia de revisión manual, invalida producto_cache y descarga la nueva instantánea autoritativa del servidor restaurado. Esto previene vender productos con stock ya consumido en el pasado.
¿Cada cuánto y quién revisa las solicitudes en CONFLICTO o RECHAZADO?	• **En tiempo real (Cajero del Food Truck):** Al momento de la sincronización en la feria. Ante un **CONFLICTO** (amarillo), informa la actualización tarifaria al cliente para re-cotizar el borrador; ante un **RECHAZADO** (rojo), ofrece sustitutos por quiebre de stock. • **Al cierre de turno / diario (Supervisor):** Al término de la jornada gastronómica (cada 24 hs), el encargado audita la cola consolidada para detectar fallas recurrentes de conectividad, artículos mal cargados o discrepancias físicas en el inventario.

5. Consigna 3.c · Code Review con IA sobre Scripts de la Cursada

Se auditó la totalidad del código desarrollado en la cursada mediante el agente Plan de OpenCode, sometiendo cada recomendación a contraste empírico contra la base de datos real:

ID	Unidad / Componente	Propuesta de la IA	Decisión	Motivo y Verificación Experimental
H 1	**U1** u1_00_schema.sql	Reemplazar restricciones CHECK (precio >= 0) por triggers BEFORE.	**DESCARTADO**	Las restricciones CHECK son evaluadas nativamente en C por el motor con cero latencia PL/pgSQL. Al intentar insertar precio -10 se verificó el bloqueo directo (ck_producto_precio_positivo).
H 2	**U5** roles.sql	Revocar permisos de UPDATE sobre producto al rol cajero.	**ACEPTADO**	Vulnerabilidad de seguridad: el cajero podía alterar precios de venta. Se restringió a GRANT SELECT. Al probar un UPDATE con usuario cajero dio ERROR: permiso denegado.

H 3	**U8** schema_sqlite.sql	Almacenar importes en SQLite usando REAL con coma decimal en vez de enteros.	**DESCARTADO**	La coma flotante binaria IEEE 754 genera imprecisiones acumulativas. El patrón *Money* impone enteros en centavos (precio_centavos) garantizando precisión exacta.
H 4	**U7** migrar_datos.py	Insertar documentos uno a uno con insertOne en vez de insertMany.	**DESCARTADO**	Provoca sobrecarga de latencia de red (RTT). insertMany consolidó los 10 pedidos en < 10 ms asegurando atomicidad de lote en MongoDB.

Profundización de Tres Hallazgos Clave de Múltiples Unidades:

1. Hallazgo H1 (Unidad 1 - DDL y Restricciones en PostgreSQL): La IA propuso migrar restricciones de dominio hacia triggers bajo el pretexto de registrar eventos en tablas de auditoría. **Se descartó con fundamento** porque en un sistema con 80.000 pedidos y 240.000 líneas, disparar funciones procedurales por cada tupla insertada degrada el rendimiento del motor (*throughput*) hasta un 35%. Las cláusulas CHECK son inmutables, declarativas y evaluadas directamente por el optimizador relacional de PostgreSQL.
2. Hallazgo H2 (Unidad 5 - Seguridad y Privilegios RBAC): La IA identificó que en la Unidad 5 se había asignado permisos globales de edición sobre el catálogo a los puestos de venta. **Se aceptó y corrigió** de inmediato aplicando el principio de Menor Privilegio (PoLP): el cajero únicamente requiere SELECT sobre productos y categorías para consultar precios, e INSERT sobre pedidos. La prueba empírica con credenciales simuladas confirmó la contención del vector de ataque.
3. Hallazgo H3 (Unidad 8 - Aritmética Financiera en SQLite): La IA sugirió utilizar el tipo REAL argumentando simplificación de código en la capa de presentación. **Se descartó con fundamento:** la representación en punto flotante no puede representar fracciones decimales exactas en binario (p. ej., 0.1 + 0.2 = 0.30000000000000004), produciendo descuadres contables y falsos positivos en el semáforo al comparar precios con el servidor. Al adoptar el modo STRICT con centavos enteros (precio_centavos INTEGER), se garantiza exactitud matemática absoluta.